CS5013: Programming with AI — Principles & Practices Document

Project: GUI Framework for Multi-Tool Panoramic Image Segmentation | **Student:** Rathan Balaji P R (CS26E009)
Stakeholder / Instructor: Dr. Sukhendu Das | Department of Computer Science & Engineering, IIT Madras

1. Executive Summary & Architectural Overview

Panoramic image segmentation introduces severe geometric challenges: 360° cylindrical equirectangular distortion, aspect ratios up to 12:1, and heterogeneous tool runtimes (Python/PyTorch, REST APIs, C++). To unify these tools, we engineered a modular Java 21 + JavaFX desktop framework adhering to clean architecture. Core components include: (i) **Model Isolation Boundary:** heavy deep-learning dependencies execute in isolated external processes via Java ProcessBuilder; (ii) **Canonical Normalization:** outputs map into a uniform SegmentationResult schema without losing unmapped metadata; (iii) **Panorama-Aware View:** interactive 360° horizontal modulo yaw wrap-around; and (iv) **Metrics Engine:** evaluating Pixel Accuracy, mIoU, and Dice score against Ind-PanSeg ground truth.

2. Prompt Engineering Principles & Lifecycle Application

Throughout this project, prompt engineering was treated as a rigorous software specification discipline rather than ad-hoc chat. We systematically applied five core principles:

- **A. Role-Framing & System Identity:** Each interaction explicitly designated the AI as a Principal Software Architect paired with a Graduate Student at IIT Madras. This anchored the model to academic rigor, strict design pattern compliance (GoF Adapter, Strategy, Observer), and zero tolerance for stub code.
- **B. Context Bounding & In-Context Grounding:** Rather than allowing the model to hallucinate specifications, prompts provided authoritative technical anchors: the CS5013 Project Proposal, the Project Design Document, and the ICVGIP 2026 research paper (*PanoFusion*, Submission 214). File paths and exact method contracts were bounded.
- **C. Negative Prompting & Anti-Pattern Constraints:** Prompts strictly enforced boundaries: *'Do NOT embed Python or CUDA inside JVM'*, *'Do NOT silently drop unsupported tool metadata'*, *'Do NOT invent arbitrary classes; use exact Ind-PanSeg 21 classes'*, and *'Do NOT use third-party CSS; use clean JavaFX CSS'*.
- **D. Output Structuring & Few-Shot Schemas:** We enforced exact JSON schemas for tool interchange (e.g. tools.json) and strict POJO Builder contracts (SegmentationResult.Builder). This ensured bidirectional serialization without contract drift.
- **E. Iterative Chain-of-Thought & Stepwise Verification:** Complex implementations were broken into discrete verified increments: Domain Model → Adapters → Tests → UI Rendering. Each step was verified through test execution before progressing to the next layer.

3. Debugging: Structured Approach & Issue Resolution

Debugging across a hybrid JavaFX-Python-CUDA environment requires a rigorous scientific methodology to isolate runtime boundaries:

- **1. Boundary Isolation (JVM vs Subprocess):** When model execution failed, we decoupled the Java adapter from the Python script. Python runners were verified standalone via CLI with known sample panoramas before connecting the Java ProcessModelAdapter, preventing cross-boundary diagnostic confusion.
- **2. Deadlock-Free Stream Consumption:** A critical bug in external process execution is OS pipe deadlock when standard output/error buffers fill. We implemented separate worker threads consuming stdout and stderr concurrently using BufferedReader, preventing Java execution freezes.
- **3. Deterministic Mock Test Baselines:** To debug rendering and evaluation logic without GPU/network latency, we created DeterministicTestAdapter. It synthesizes mathematically predictable horizontal wrap-around regions, proving the metric formulas and modulo yaw rotation independently.
- **4. Test-Driven Reproducers:** Whenever a bug emerged (e.g. schema mismatches when switching to 21-class Ind-PanSeg), we authored a focused failing test in AdapterTest.java, debugged the failure trace, updated the normalization parser, and re-ran tests until green.

4. Version Control (Git): Code & Prompt Lifecycle Tracking

Version control was leveraged not merely as an archive, but as an auditable trail of architectural decisions, code evolutions, and AI prompts:

- **A. Conventional Atomic Commits:** We adhered strictly to the Conventional Commits specification (feat:, test:, docs:, refactor:). Every commit represents an atomic, compiling state with passing test suites. Examples: feat: implement core domain model, test: add 35 comprehensive JUnit 5 tests, feat: integrate PanoFusion and Ind-PanSeg 21-class models from ICVGIP 2026 paper.
- **B. Bidirectional Prompt-to-Commit Tracking:** Commit messages explicitly link functional changes to prompt specifications. In addition, the complete prompt-response trajectories are preserved in versioned Markdown artifacts and transcript logs, ensuring full reproducibility and auditability required by the course guidelines.
- **C. Build Hygiene & Configuration Versioning:** The Maven wrapper (mvnw.cmd), dependencies, and external tool definitions (config/tools.json) are strictly version-controlled. Local machine-specific artifacts (e.g. .m2 cache, OS dumps, compiled bytecode) are isolated via an exhaustive .gitignore.
- **D. Linear, Rebase-Friendly History:** We maintain a clean linear Git history without redundant merge bubbles, allowing seamless rollback and crystal-clear delta reviews for TAs.

5. Testing, Refactoring & Documentation Standards

High code quality and pedagogical maintainability were enforced through multi-tier testing, design pattern refactoring, and comprehensive documentation:

Test Category	Test Classes & Target	Coverage & Verification Target
Unit Tests	MetricsCalculatorTest, ResultNormalizerTest, SegmentationModelTest	Verified mathematical correctness of Pixel Accuracy, mIoU, Dice; JSON schema parser; boundary wrap logic.
Boundary & Execution Tests	ExecutionBoundaryTest, ValidationServiceTest	Tested timeout enforcement, invalid aspect ratio handling, missing tools, and stream buffer isolation.
Integration & UI Tests	AdapterTest, AsyncCoordinatorTest, GuiWorkflowTest, EndToEndSystemTest	Complete asynchronous multi-tool pipeline: image validation → Python execution → normalization → wrap-around view → metrics.

- **Refactoring to GoF Design Patterns:** 1. **Adapter Pattern:** ProcessModelAdapter and HttpModelAdapter adapt heterogeneous external protocols to SegmentationTool.
2. **Strategy Pattern:** Metrics calculation and image rendering algorithms are decoupled from model execution.
3. **Observer/Listener Pattern:** AsyncSegmentationCoordinator publishes progress events to GUI listeners without blocking JavaFX Application Thread.
4. **Builder Pattern:** SegmentationResult.Builder guarantees immutability and complete audit tracking.
- **Documentation Integrity:** The repository features a professional README.md, an authoritative CS5013 Proposal and Design Document, comprehensive Javadoc on all public API classes, and an automated demonstration report (demonstration_results.md) illustrating real visual comparison overlays and 360 wrap-around views.

6. Appendix: Concrete Sample Prompts & Prompt Engineering Analysis

Below are actual prompts employed across the development lifecycle, annotated with the engineering principles applied:

Sample Prompt 1: Initial Architecture Scaffolding & Contract Enforcement

```
Role & Context: You are a Principal Java/Software Architect implementing the CS5013 Design Document for IIT Madras.
Task: Implement the complete domain model and adapter contracts in Java 21.
Constraints:
1. Model classes (SegmentationResult, MaskData, SemanticClass) must be strictly immutable using Builder pattern.
2. External models must NEVER run in the JVM; design ProcessModelAdapter using ProcessBuilder with background
stdout/stderr threads.
3. If external models return unrecognized metadata attributes, do NOT drop them. Store them in an unsupportedAttributes
Map.
4. Provide JUnit 5 unit tests verifying all model constraints.
Engineering Principle Demonstrated: Role-framing + Negative Prompting (preventing JVM pollution) + Concrete Contract
Schemas.
```

Sample Prompt 2: Cylindrical Wrap-Around Rendering & Boundary Continuity

```
Context: The input is an equirectangular 360° panorama. Standard 2D viewers fail because objects split across 0° and 360°
appear severed.
Task: Create PanoramaImageRenderer in JavaFX to support interactive horizontal yaw rotation.
Algorithm Requirement:
Implement cyclic modulo offset: sourceX = (x + offsetPixels) % width.
Ensure seamless interpolation across the seam. Write a unit test verifying that pixel labels at x=0 and x=width-1 retain
geometric continuity.
Engineering Principle Demonstrated: Algorithmic Precision + Domain Geometry Grounding + Test-Driven Specification.
```

Sample Prompt 3: Integrating ICVGIP 2026 Paper Benchmark Models

```
Context: Research paper: 'PanoFusion: Boundary-Aware Vision-Language Segmentation for Stitched Indoor Panoramas' (ICVGIP
2026).
Task: Integrate the exact models from Table 3 into config/tools.json and external runners:
1. panofusion: CLIP + DAN + SBR-Net MobileSAM boundary refinement (54.94% mIoU).
2. ops-baseline: Coarse vision-language backbone without SBR-Net (44.12% mIoU).
3. segformer-baseline: Perspective transformer fine-tuned with focal loss (31.35% mIoU).
4. Adopt the authoritative 21 classes of Ind-PanSeg (Table 2: Roof, Bed, Wall, Floor, Monitor, Chair, Table, etc.).
5. Update test assertions and verify all 35 tests pass with zero regressions.
Engineering Principle Demonstrated: Zero Code Modification to Core Architecture (Pure Adapter Pattern Extension) + Real
Benchmark Grounding.
```

Student Declaration: I confirm that all code, tests, and documentation reflect my own architecture and understanding, guided by software engineering best practices taught in CS5013. AI tools were utilized strictly in compliance with course policy.